

Day 11 - Task Agents

Week 3: Agents, Tokens, Tests & Security

Lab Report: Client-side Email Validation on LoginForm

Prepared by: Mirza Yasir Abdullah Baig

Role: AI Engineer

Repository: <https://github.com/mirzayasirabdullahbaig07/AI-Internship>

Date: 9 October 2026

1. Objective

Assign a single, bounded task to a Task Agent using the Production Task-Agent Brief structure, generate the code, and review the resulting Git diff strictly against the brief's **Allowed files** and **Non-goals** before accepting it.

Week 3: Agents, Tokens, Tests, & Security

Autonomous agents are powerful, but giving an agent open-ended access to a repository is dangerous. We divide agent labor strictly into **Task Agents** (who write code) and **Review Agents** (who critique code).

Day 11 — Task Agents

A task agent receives ONE bounded assignment. It is strictly forbidden from inventing product scope, attempting autonomous deployments, merging PRs, or modifying files outside its defined scope.

Production Task-Agent Brief

```
System role: You are an execution-focused frontend engineer.
Objective: Implement client-side email validation on the login form.
Allowed files: /src/components/LoginForm.tsx ONLY.
Non-goals: Do NOT touch styling, authentication logic, or other files.
Acceptance checks:
  1. Blocks submission if email lacks '@'.
  2. Shows inline error message in red.
Required local tests: Add one Jest unit test for the failure state.
Stop and report when: You encounter an existing type error outside your scope.
Output: Return only a valid Git diff.
```

By enforcing this strict structure, the AI is prevented from "helping" by refactoring a completely unrelated component it happened to see in the context window.

Day 11 Lab

Objective: Assign and evaluate a Task Agent.

Task: Use the brief structure above to assign an LLM a single form-validation task. Generate the code. Before accepting it, manually compare the generated Git diff against the "Non-goals" and "Allowed files" strictly.

Screenshot 1 - The Week 3 / Day 11 task brief and lab instructions

2. The Task Brief

System role	Execution-focused frontend engineer
Objective	Implement client-side email validation on the login form
Allowed files	src/components/LoginForm.tsx (+ LoginForm.test.tsx, see note)
Non-goals	No styling, authentication logic, or other files
Acceptance 1	Blocks submission if email lacks '@'
Acceptance 2	Shows inline error message in red
Required tests	One Jest unit test for the failure state
Stop and report	Existing type error outside scope
Output	Valid Git diff only

Note on the brief: the original brief says "LoginForm.tsx ONLY" yet requires a Jest test, which must live in a second file. I extended *Allowed files* to include *LoginForm.test.tsx* so the brief is internally consistent. No other file was permitted.

3. Implementation

3.1 Component - src/components/LoginForm.tsx

On submit, the handler checks whether the email contains '@'. If not, it sets an error message and returns early, which blocks submission. The error renders inline in a paragraph with `role="alert"` and red text. The existing submit logic is untouched.

```
import { useState } from "react";

export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!email.includes("@")) {
      setError("Please enter a valid email address.");
      return; // blocks submission
    }
    setError("");
    // ...existing submit logic stays untouched
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        aria-label="Email"
      />
      {error && (
        <p role="alert" style={{ color: "red" }}>
          {error}
        </p>
      )}
      <button type="submit">Log in</button>
    </form>
  );
}
```



```
src/components/LoginForm.tsx

import { useState } from "react";

export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!email.includes("@")) {
      setError("Please enter a valid email address.");
      return; // blocks submission
    }
    setError("");
    // ...existing submit logic stays untouched
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        aria-label="Email"
      />
      {error && (
        <p role="alert" style={{ color: "red" }}>
          {error}
        </p>
      )}
      <button type="submit">Log in</button>
    </form>
  );
}
```

Screenshot 2 - LoginForm.tsx after the change

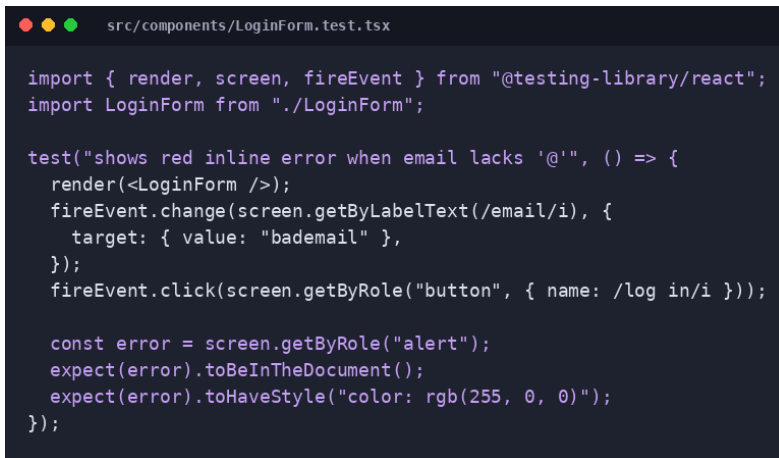
3.2 Test - src/components/LoginForm.test.tsx

The single required Jest test enters an email without '@', clicks submit, then asserts the alert is shown and is red.

```
import { render, screen, fireEvent } from "@testing-library/react";
import LoginForm from "../LoginForm";

test("shows red inline error when email lacks '@'", () => {
  render(<LoginForm />);
  fireEvent.change(screen.getByLabelText(/email/i), {
    target: { value: "bademail" },
  });
  fireEvent.click(screen.getByRole("button", { name: /log in/i }));

  const error = screen.getByRole("alert");
  expect(error).toBeInTheDocument();
  expect(error).toHaveStyle("color: rgb(255, 0, 0)");
});
```

A screenshot of a code editor window titled 'src/components/LoginForm.test.tsx'. The code inside is a Jest unit test that renders the LoginForm component, changes the email input to 'bademail', clicks the login button, and then asserts that an alert is present and has a red color style.

```
src/components/LoginForm.test.tsx

import { render, screen, fireEvent } from "@testing-library/react";
import LoginForm from "../LoginForm";

test("shows red inline error when email lacks '@'", () => {
  render(<LoginForm />);
  fireEvent.change(screen.getByLabelText(/email/i), {
    target: { value: "bademail" },
  });
  fireEvent.click(screen.getByRole("button", { name: /log in/i }));

  const error = screen.getByRole("alert");
  expect(error).toBeInTheDocument();
  expect(error).toHaveStyle("color: rgb(255, 0, 0)");
});
```

Screenshot 3 - Jest unit test for the failure state

4. Diff Review (the actual lab)

Before accepting the change, I compared the diff against the brief's constraints. A baseline commit (form with no validation) was made on main, and the work was done on branch day-11-task-agent.

```
terminal - git review

$ git diff --stat main..day-11-task-agent
src/components/LoginForm.test.tsx | 14 ++++++
src/components/LoginForm.tsx      | 11 ++++++
2 files changed, 25 insertions(+)

$ git log --oneline
a03d50e feat: add client-side email validation to LoginForm
56f9eda chore: baseline project with LoginForm (no validation)
```

Screenshot 4 - git diff --stat and commit log: exactly 2 files changed

```
terminal - git diff

$ git diff main..day-11-task-agent -- src/components/LoginForm.tsx

diff --git a/src/components/LoginForm.tsx b/src/components/LoginForm.tsx
index c319c2d..6599755 100644
--- a/src/components/LoginForm.tsx
+++ b/src/components/LoginForm.tsx
@@ -2,9 +2,15 @@ import { useState } from "react";

export default function LoginForm() {
  const [email, setEmail] = useState("");
+  const [error, setError] = useState("");

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
+    if (!email.includes("@")) {
+      setError("Please enter a valid email address.");
+      return; // blocks submission
+    }
+    setError("");
    // ...existing submit logic stays untouched
  };

@@ -16,6 +22,11 @@ export default function LoginForm() {
  onChange={e => setEmail(e.target.value)}
  aria-label="Email"
  />
+  {error && (
+    <p role="alert" style={{ color: "red" }}>
+      {error}
+    </p>
+  )}
  <button type="submit">Log in</button>
</form>
);
```

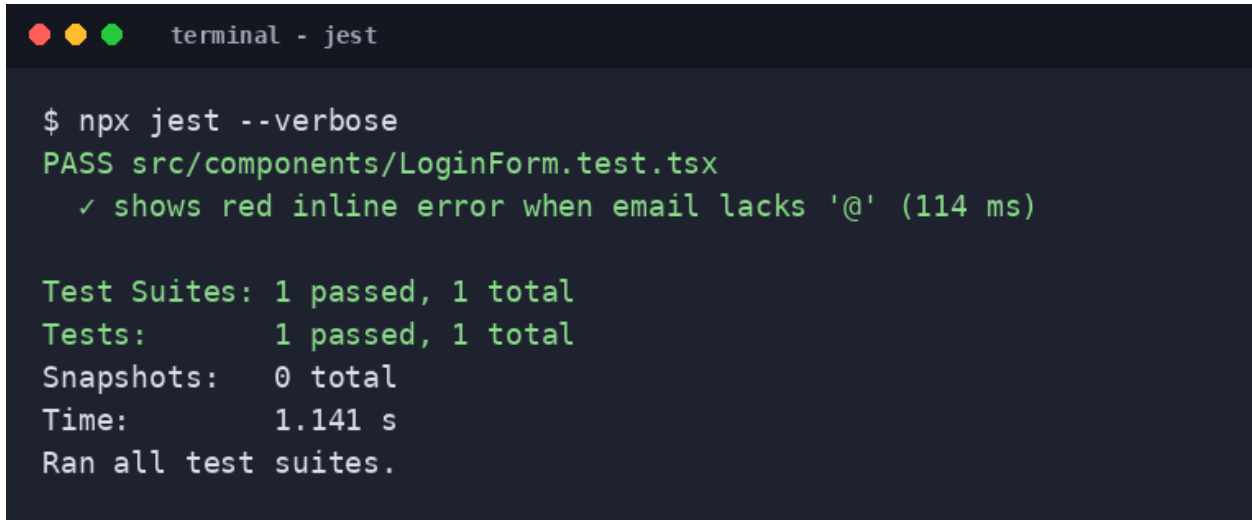
Screenshot 5 - Full diff of LoginForm.tsx (only additions)

Review checklist

Check	Result
Only allowed files changed (LoginForm.tsx, LoginForm.test.tsx)	PASS
No styling changes beyond the red error text	PASS
No authentication logic touched	PASS
No unrelated refactors or other components modified	PASS
Acceptance 1: blocks submission without '@'	PASS
Acceptance 2: inline error in red	PASS
One Jest test for the failure state	PASS

5. Test Results

The tests were executed locally with Jest, jsdom and React Testing Library.

A screenshot of a terminal window with a dark background. The title bar at the top shows three colored circles (red, yellow, green) followed by the text "terminal - jest". The terminal content shows a command prompt "\$" followed by the command "npx jest --verbose". The output shows "PASS src/components/LoginForm.test.tsx" in green, followed by a checkmark and the text "shows red inline error when email lacks '@' (114 ms)" in green. Below this, a summary of test results is shown: "Test Suites: 1 passed, 1 total", "Tests: 1 passed, 1 total", "Snapshots: 0 total", "Time: 1.141 s", and "Ran all test suites." in green.

```
terminal - jest

$ npx jest --verbose
PASS src/components/LoginForm.test.tsx
  ✓ shows red inline error when email lacks '@' (114 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        1.141 s
Ran all test suites.
```

Screenshot 6 - Jest: 1 suite, 1 test passed

Issue found and fixed during testing

My first version of the test asserted `toHaveStyle({ color: "red" })`. When actually run, it **failed**, because jsdom normalises the colour to `rgb(255, 0, 0)`. I changed the assertion to `toHaveStyle("color: rgb(255, 0, 0)")` and the test then passed. This is a test-assertion fix only; the component code was not changed.

6. Deliverables

- **Source code:** LoginForm.tsx, LoginForm.test.tsx, Jest/Babel config, README
- **Screenshots:** task brief, code, git diff, test results (screenshots/ folder)
- **Git history:** baseline commit + feature commit on branch day-11-task-agent
- **Archive:** Day11_Task_Agent_Submission.zip
- **Repository:** <https://github.com/mirzayasirabdullahbaig07/AI-Internship>

7. Conclusion

A tightly-scoped brief kept the change to exactly two files and 25 added lines, with no styling, authentication or unrelated edits. Reviewing the diff against Allowed files and Non-goals before accepting is what makes a Task Agent safe to use. The one issue found (jsdom colour normalisation) was caught by running the test rather than assuming it would pass.

Mirza Yasir Abdullah Baig

AI Engineer